

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 5
CLEAN CODE DAN DEFENSIVE PROGRAMMING**

**DISUSUN OLEH :
BAGUS DWI PUTRA SETIAWAN - 2350081042**



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR	iv
BAB I. HASIL PRAKTIKUM.....	1
I.1 Latihan Clean Code (Meaningful Names).....	1
I.1.A Langkah Kerja / Source Code	1
I.1.B Screenshoot	2
I.1.C Analisis.....	3
I.2 Latihan Clean Code (Use Pronounceable Names)	3
I.2.A Langkah Kerja / Source Code	3
I.2.B Screenshoot	4
I.2.C Analisis.....	4
I.3 Latihan Clean Code (Switch Statements).....	5
I.3.A Langkah Kerja / Source Code	5
I.3.B Screenshoot	6
I.3.C Analisis.....	7
I.4 Latihan Clean Code (Data Abstraction)	7
I.4.A Langkah Kerja / Source Code	7
I.4.B Screenshoot	8
I.4.C Analisis.....	9
I.5 Latihan Clean Code (Comments)	9
I.5.A Langkah Kerja / Source Code	9

I.5.B	Screenshoot	12
I.5.C	Analisis.....	12
I.6	Latihan Clean Code (Formatting).....	13
I.6.A	Langkah Kerja / Source Code	13
I.6.B	Screenshoot	14
I.6.C	Analisis.....	15
I.7	Latihan Defensive Programming (Assertion).....	15
I.7.A	Langkah Kerja / Source Code	15
I.7.B	Screenshoot	16
I.7.C	Analisis.....	17
I.8	Latihan Defensive Programming (Single Catch Block).....	17
I.8.A	Langkah Kerja / Source Code	17
I.8.B	Screenshoot	18
I.8.C	Analisis.....	18
I.9	Latihan Defensive Programming (Multiple Catch Block)	18
I.9.A	Langkah Kerja / Source Code	18
I.9.B	Screenshoot	20
I.9.C	Analisis.....	20
I.10	Latihan Defensive Programming (The Finally Block).....	20
I.10.A	Langkah Kerja / Source Code	20
I.10.B	Screenshoot	21
I.10.C	Analisis.....	22
BAB II.	TUGAS PRAKTIKUM	23
II.1	Tugas Clean Code (Naming)	23

II.1.A	Langkah Kerja / Source Code	23
II.1.B	Screenshoot	24
II.1.C	Analisis.....	24
II.2	Tugas Clean Code (Formatting)	25
II.2.A	Langkah Kerja / Source Code	25
II.2.B	Screenshoot	26
II.2.C	Analisis.....	27
II.3	Tugas Defensive Programming (nomor 3).....	27
II.3.A	Langkah Kerja / Source Code	27
II.3.B	Screenshoot	29
II.3.C	Analisis.....	29
II.4	Tugas Defensive Programming (nomor 4).....	30
II.4.A	Langkah Kerja / Source Code	30
II.4.B	Screenshoot	35
II.4.C	Analisis.....	36
BAB III.	KESIMPULAN.....	38

DAFTAR GAMBAR

Gambar 1. 1 Code Sebelum Diperbaiki	2
Gambar 1. 2 Code Sesudah Diperbaiki	2
Gambar 1. 3 Code Sebelum Diperbaiki	4
Gambar 1. 4 Code Sesudah Diperbaiki	4
Gambar 1. 5 Code Sebelum Diperbaiki	6
Gambar 1. 6 Code Sesudah Diperbaiki	7
Gambar 1. 7 Code Sebelum Diperbaiki	8
Gambar 1. 8 Code Sesudah Diperbaiki	9
Gambar 1. 9 Bad Comments	12
Gambar 1. 10 Good Comments.....	12
Gambar 1. 11 Bad Formatting.....	14
Gambar 1. 12 Good Formatting	15
Gambar 1. 13 Output Assertion	16
Gambar 1. 14 Single Catch Block.....	18
Gambar 1. 15 Multiple Catch Block	20
Gambar 1. 16 The Finally Block.....	21
Gambar 2. 1 Code Sebelum Diperbaiki	24
Gambar 2. 2 Code Sesudah Diperbaiki	24
Gambar 2. 3 Code Sebelum Diperbaiki	26
Gambar 2. 4 Code Sesudah Diperbaiki	27
Gambar 2. 5 Code Sebelum Diperbaiki	29

Gambar 2. 6 Code Sesudah Diperbaiki	29
Gambar 2. 7 Code Sebelum Diperbaiki	35
Gambar 2. 8 Code Sesudah Diperbaiki	36
Gambar 2. 9 Ouput Code Setelah Diperbaiki.....	36

BAB I. HASIL PRAKTIKUM

I.1 Latihan Clean Code (Meaningful Names)

I.1.A Langkah Kerja / Source Code

Sebelum

```
import java.util.ArrayList;
import java.util.List;

public class cleancode1 {
    public List<int[]> getThem() {
        List<int[]> list1 = new ArrayList<int[]>();
        for (int[] x : theList)
            if (x[0] == 4)
                list1.add(x);
        return list1;
    }
}
```

Sesudah

```
import java.util.ArrayList;
import java.util.List;

public class cleancode1 {
    public final static int STATUS_VALUE = 0;
    public final static int FLAGGED = 4;

    public List2<int[]> getFlaggedCells() {
```

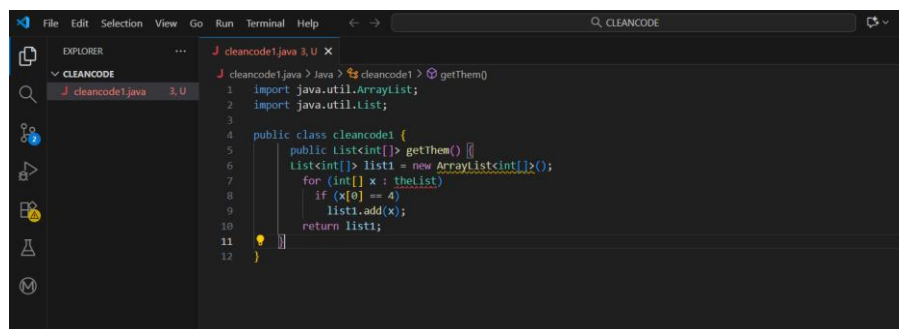
```

        List2<int[]> flaggedCells = new
        ArrayList<int[]>();

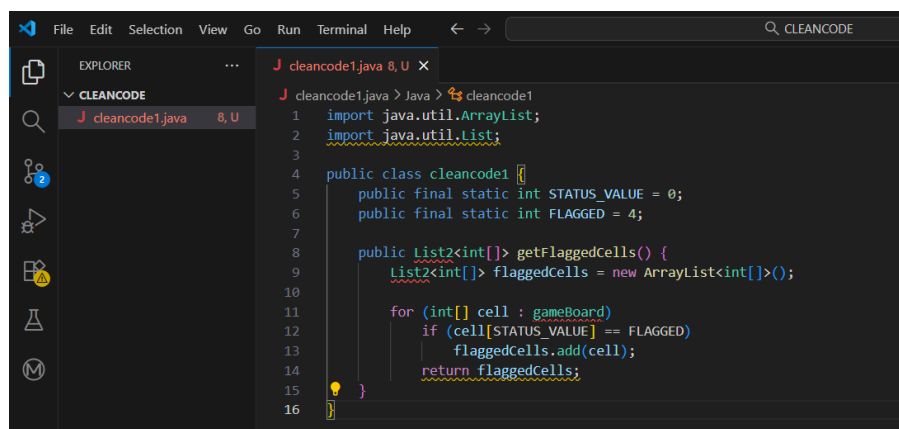
        for (int[] cell : gameBoard)
            if (cell[STATUS_VALUE] == FLAGGED)
                flaggedCells.add(cell);
        return flaggedCells;
    }
}

```

I.1.B Screenshoot



Gambar 1. 1 Code Sebelum Diperbaiki



Gambar 1. 2 Code Sesudah Diperbaiki

I.1.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu melakukan perbaikan dengan memberikan nama variabel dan metode yang lebih bermakna sehingga kode lebih mudah dipahami. Pada versi awal, nama seperti list1, x, dan theList tidak menggambarkan fungsi sebenarnya, sehingga membingungkan pembaca.

Setelah diperbaiki, nama seperti FLAGGED, STATUS_VALUE, flaggedCells, dan getFlaggedCells() membuat maksud kode jauh lebih jelas dan mudah diikuti. Selain itu, penggunaan konstanta untuk angka magic number (seperti 4) meningkatkan kualitas kode karena memudahkan pemeliharaan dan mengurangi risiko kesalahan.

I.2 Latihan Clean Code (Use Pronounceable Names)

I.2.A Langkah Kerja / Source Code

Sebelum

```
import java.util.Date;

class DtaRcrd102 {
    private Date genymdhms;
    private Date modymdhms;
    private final String pszqint = "102";
    /* ... */
}
```

Sesudah

```
import java.util.Date;

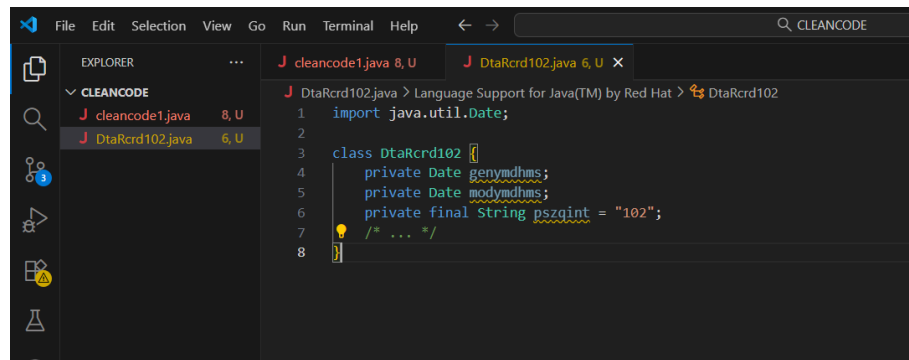
class Customer {
    private Date generationTimestamp;
```

```

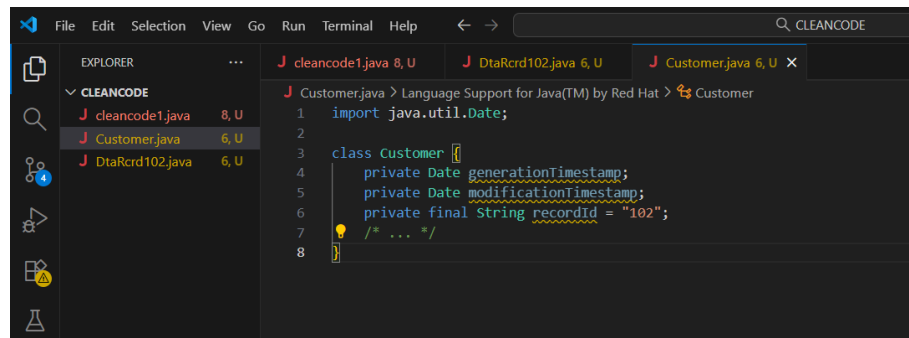
private Date modificationTimestamp;
private final String recordId = "102";
/* ... */
}

```

I.2.B Screenshoot



Gambar 1. 3 Code Sebelum Diperbaiki



Gambar 1. 4 Code Sesudah Diperbaiki

I.2.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menekankan pentingnya penggunaan nama yang mudah diucapkan dan dipahami. Pada versi awal, nama seperti DtaRcrd102, genymdhms, dan pszqint sulit dibaca dan tidak memiliki arti yang jelas, sehingga menghambat pemahaman programmer lain.

Setelah diperbaiki, class Customer, serta variabel seperti generationTimestamp, modificationTimestamp, dan recordId jauh lebih intuitif. Dengan nama-nama yang mudah diucapkan dan bermakna, proses komunikasi antar developer menjadi lebih efektif, terutama saat melakukan diskusi atau code review.

I.3 Latihan Clean Code (Switch Statements)

I.3.A Langkah Kerja / Source Code

Sebelum

```
import java.util.ArrayList;
import java.util.List;

public class cleancode1 {
    public List<int[]> getThem() {
        List<int[]> list1 = new ArrayList<int[]>();
        for (int[] x : theList)
            if (x[0] == 4)
                list1.add(x);
        return list1;
    }
}
```

Sesudah

```
import java.util.ArrayList;
import java.util.List;

public class cleancode1 {
    public final static int STATUS_VALUE = 0;
    public final static int FLAGGED = 4;
```

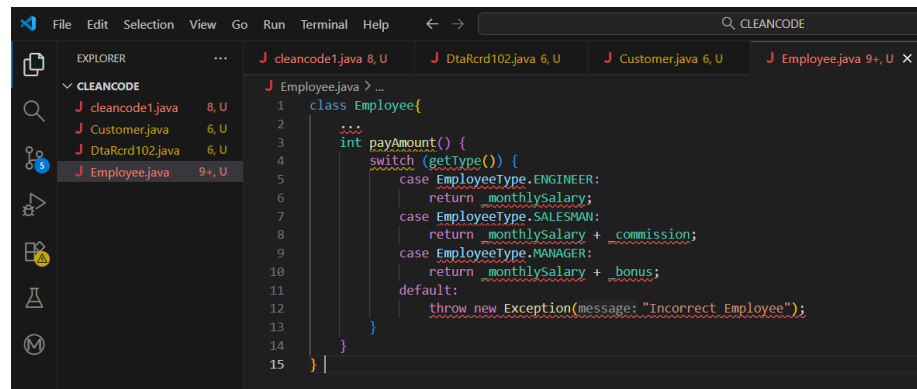
```

        public List2<int[]> getFlaggedCells() {
            List2<int[]> flaggedCells = new
ArrayList<int[]>();

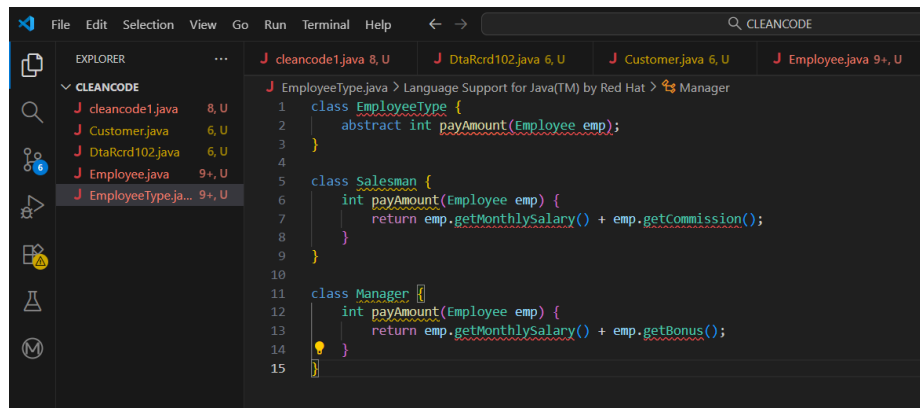
            for (int[] cell : gameBoard)
                if (cell[STATUS_VALUE] == FLAGGED)
                    flaggedCells.add(cell);
            return flaggedCells;
        }
    }
}

```

I.3.B Screenshoot



Gambar 1. 5 Code Sebelum Diperbaiki



Gambar 1. 6 Code Sesudah Diperbaiki

I.3.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu memperbaiki penggunaan struktur kontrol agar lebih jelas dan mudah dipahami. Versi awal menggunakan nama variabel yang tidak informatif, sehingga logika yang melibatkan pengecekan nilai array menjadi sulit dilacak.

Setelah diperbaiki, kode memiliki struktur yang lebih jelas dengan nama seperti gameBoard, STATUS_VALUE, dan FLAGGED, sehingga alur pengambilan keputusan menjadi lebih transparan. Dengan penamaan yang tepat, logika switch atau pengecekan kondisi menjadi lebih ringkas dan bebas dari magic number.

I.4 Latihan Clean Code (Data Abstraction)

I.4.A Langkah Kerja / Source Code

Sebelum

```

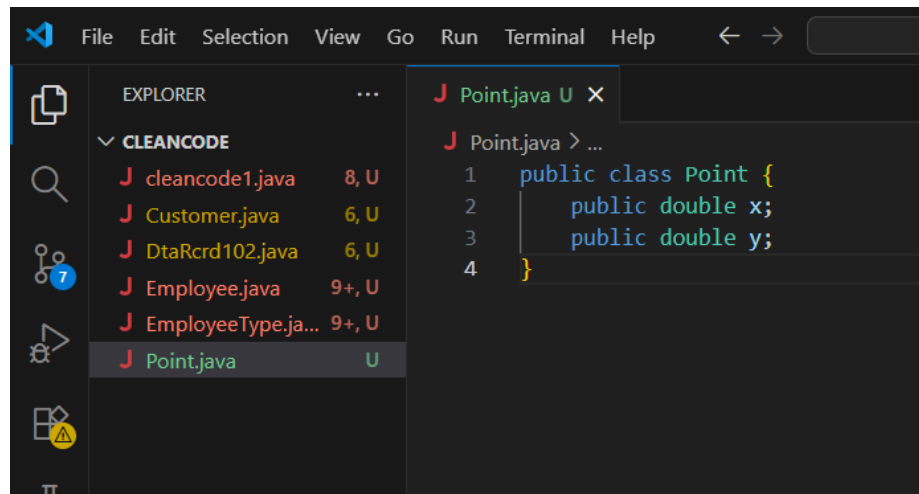
public class Point {
    public double x;
    public double y;
}

```

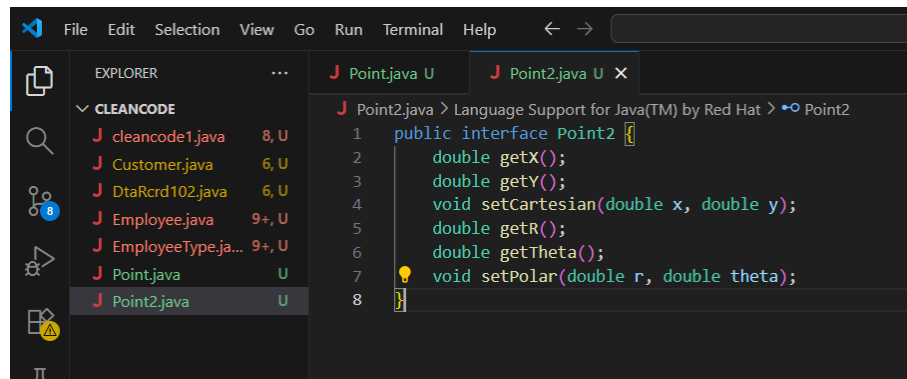
Sesudah

```
public interface Point2 {  
    double getX();  
    double getY();  
    void setCartesian(double x, double y);  
    double getR();  
    double getTheta();  
    void setPolar(double r, double theta);  
}
```

I.4.B Screenshoot



Gambar 1. 7 Code Sebelum Diperbaiki



Gambar 1. 8 Code Sesudah Diperbaiki

I.4.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menekankan pentingnya abstraksi data untuk menghindari manipulasi langsung terhadap atribut. Pada versi awal, class Point memiliki atribut publik sehingga dapat diubah secara langsung tanpa kontrol.

Setelah diperbaiki, struktur dibuat menjadi sebuah interface dengan metode akses seperti getX(), setCartesian(), dan setPolar(), yang memberikan lapisan abstraksi jauh lebih baik. Pendekatan ini memperkuat prinsip encapsulation dan menghindari ketergantungan langsung pada representasi internal objek.

I.5 Latihan Clean Code (Comments)

I.5.A Langkah Kerja / Source Code

Bad Comments

1. Journal Comments

```
* Changes (from 11-Oct-2001)
* -----
* 11-Oct-2001 : Re-organised the class and moved
it to new
* package com.jrefinery.date (DG);
```

```
* 05-Nov-2001 : Added a getDescription() method,  
and  
* eliminated NotableDate class (DG);  
* 12-Nov-2001 : IBD requires setDescription()  
method, now  
* that NotableDate class is gone (DG); Changed  
* getPreviousDayOfWeek(),  
* getFollowingDayOfWeek() and  
* getNearestDayOfWeek() to correct bugs (DG);  
* 05-Dec-2001 : Fixed bug in SpreadsheetDate class  
(DG);  
* 29-May-2002 : Moved the month constants into a  
separate  
* interface (MonthConstants) (DG);
```

2. Noise Comments

```
/**  
 * Default constructor.  
 */  
protected AnnualDateRule() { }  
/** The day of the month. */  
private int dayOfMonth;  
/**  
 * Returns the day of the month.  
 * @return the day of the month.  
 */  
public int getDayOfMonth() {  
    return dayOfMonth;  
}
```


Good Comments

1. Legal Comments

```
//Copyright (C) 2011 by Osoco. All rights reserved.  
//Released under the terms of the GNU General Public License  
//version 2 or later.
```

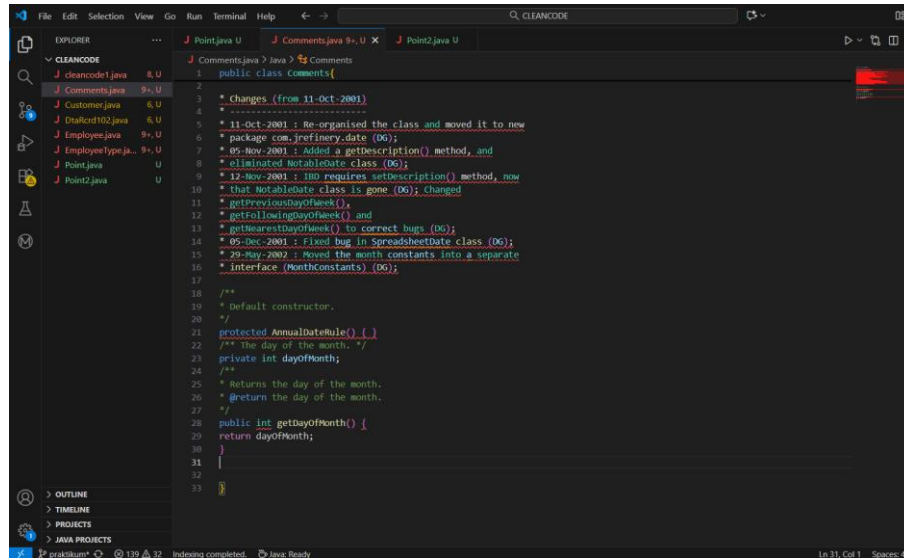
2. Informative Comments

```
//Returns an instance of the Responder being tested.  
protected abstract Responder responderInstance();  
//renaming the function: responderBeingTested  
//format matched kk:mm:ss EEE, MMM dd, yyyy  
Pattern timeMatcher =  
Pattern.compile("\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

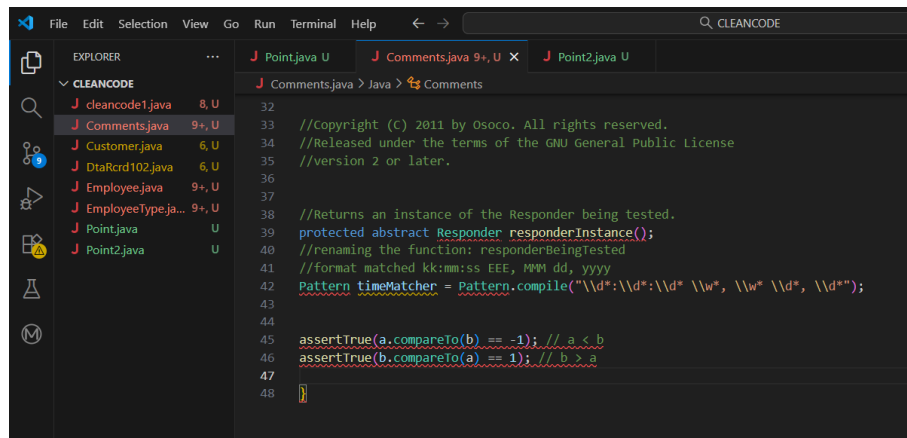
3. Clarification

```
assertTrue(a.compareTo(b) == -1); // a < b  
assertTrue(b.compareTo(a) == 1); // b > a
```

I.5.B Screenshoot



Gambar 1. 9 Bad Comments



Gambar 1. 10 Good Comments

I.5.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu membedakan antara komentar yang baik dan komentar yang buruk. Komentar buruk seperti "journal comments" dan "noise comments" cenderung memenuhi kode dengan informasi yang tidak relevan, redundant, atau tidak lagi diperlukan. Hal ini justru mengganggu pembacaan dan mengurangi kualitas kode.

Komentar yang buruk sering kali menjadi tanda bahwa kode kurang jelas dan membutuhkan perbaikan struktur. Sebaliknya, komentar yang baik memberi informasi tambahan yang tidak dapat dijelaskan oleh kode secara langsung, seperti komentar hukum, penjelasan format data, atau klarifikasi perilaku yang tidak intuitif.

I.6 Latihan Clean Code (Formatting)

I.6.A Langkah Kerja / Source Code

Bad Formatting

```
import java.io.InputStream; import
java.io.OutputStream; import java.net.Socket;
public class FitNesseExpediter implements
ResponseSender
{
    private      Socket      socket;
    private      InputStream  input;
    private      OutputStream output;
    private      Request      request;
    private      Response     response;
    private      FitNesseContext context;
    protected    long
requestParsingTimeLimit;
    private      long          requestProgress;
    private      long
requestParsingDeadline;
    private      boolean       hasError;
}
```

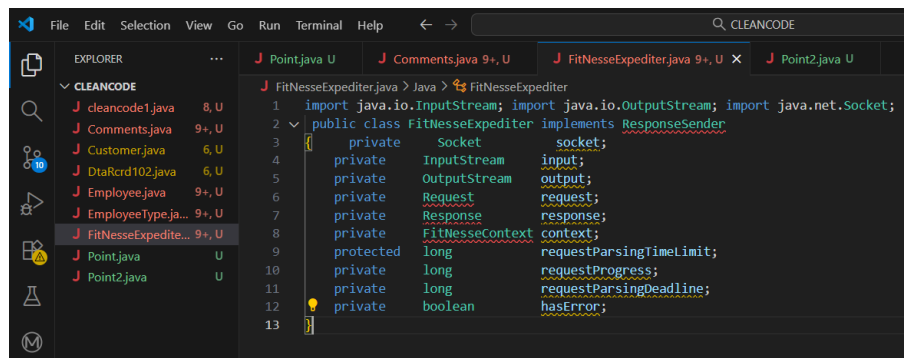
Good Formatting

```

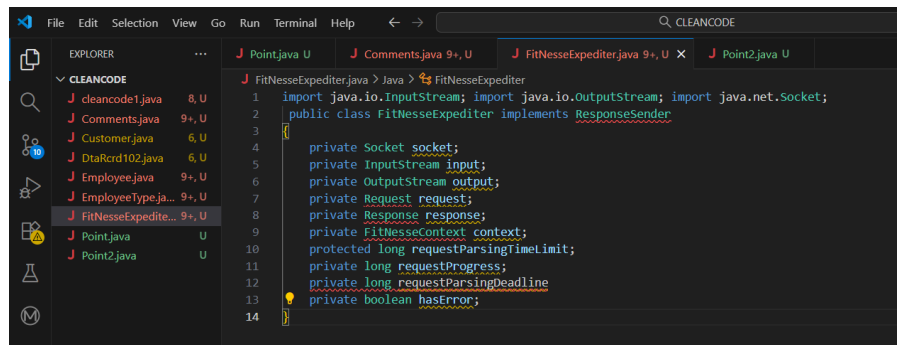
public class FitNesseExpediter implements
ResponseSender
{
    private Socket socket;
    private InputStream input;
    private OutputStream output;
    private Request request;
    private Response response;
    private FitNesseContext context;
    protected long requestParsingTimeLimit;
    private long requestProgress;
    private long requestParsingDeadline
    private boolean hasError;
}

```

I.6.B Screenshoot



Gambar 1. 11 Bad Formatting



Gambar 1. 12 Good Formatting

I.6.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu melakukan perbaikan untuk meningkatkan keterbacaan kode melalui format yang konsisten. Versi awal menempatkan deklarasi variabel dalam satu baris yang panjang dan tidak rapi, sehingga sulit dibaca dan membingungkan struktur kelas.

Setelah diperbaiki, setiap variabel dideklarasikan pada baris terpisah dengan indentasi yang seragam sehingga kode menjadi lebih mudah dipahami dan terlihat profesional. Format yang rapi juga membantu developer dalam memelihara kode serta mengurangi risiko kesalahan akibat struktur yang tidak jelas.

I.7 Latihan Defensive Programming (Assertion)

I.7.A Langkah Kerja / Source Code

Example.java

```
import java.util.Scanner;

public class Example {
    public static void main(String[] args) {
        int age = 14;
        assert age <= 18 : "Cannot Vote";
    }
}
```

```

        System.out.println("The voter's age is "
+ age);

        Scanner scanner = new Scanner( System.in
);

        System.out.print("Enter ur age ");
        int value = scanner.nextInt();
        assert value>=18:" Not valid";
        System.out.println("value is "+value);

    }
}

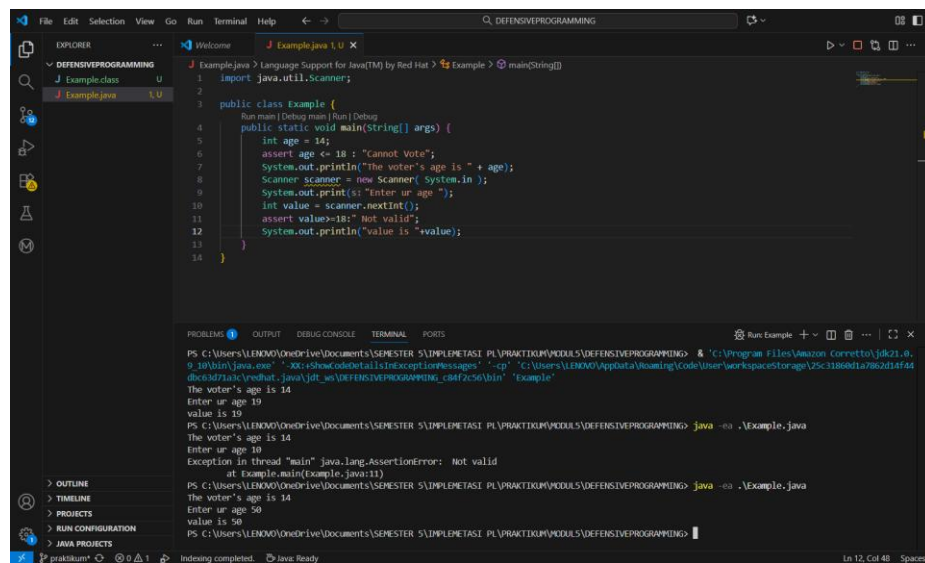
```

Jalankan pada terminal dengan perintah berikut

```
Java -ea .\Example.java
```

Lalu input angka di bawah 18, lihat apa yang terjadi.

I.7.B Screenshoot



Gambar 1.13 Output Assertion

I.7.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menggunakan assertion untuk memastikan kondisi yang wajib benar saat program berjalan. Pada kode, assert digunakan untuk memverifikasi usia pemilih bila nilai tidak memenuhi syarat logis, program menghentikan eksekusi dan memberikan pesan kesalahan.

Hal ini membantu mendeteksi error lebih awal pada tahap pengembangan, terutama untuk kondisi yang tidak boleh terjadi. Dengan menjalankan program menggunakan flag -ea, kita dapat melihat bagaimana assertion bekerja saat diberi input yang salah.

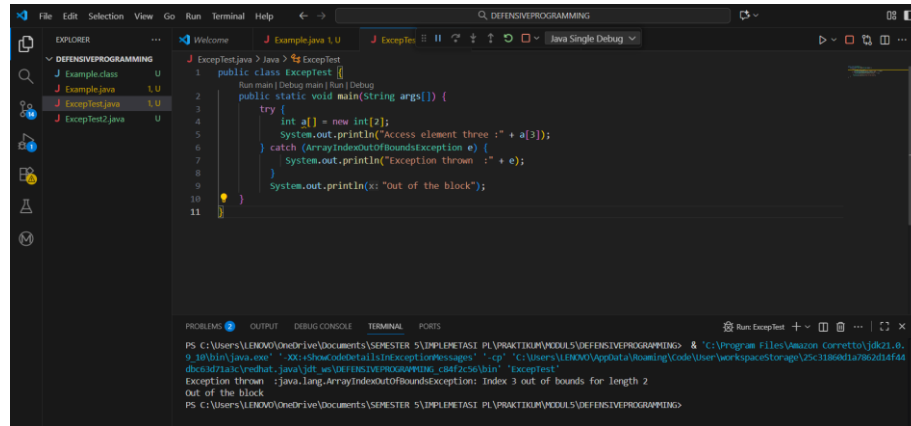
I.8 Latihan Defensive Programming (Single Catch Block)

I.8.A Langkah Kerja / Source Code

ExceptTest.java

```
public class ExceptTest {  
    public static void main(String args[]) {  
        try {  
            int a[] = new int[2];  
            System.out.println("Access element  
three :" + a[3]);  
        } catch (ArrayIndexOutOfBoundsException  
e) {  
            System.out.println("Exception thrown  
:" + e);  
        }  
        System.out.println("Out of the block");  
    }  
}
```

I.8.B Screenshoot



Gambar 1. 14 Single Catch Block

I.8.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu memperlihatkan penggunaan satu blok catch untuk menangani exception tertentu. Ketika program mencoba mengakses elemen ketiga dari array berukuran dua, exception `ArrayIndexOutOfBoundsException` langsung ditangani oleh catch block.

Hal ini menunjukkan cara sederhana untuk mencegah program crash sekaligus memberikan informasi yang jelas mengenai jenis kesalahan yang terjadi. Dengan pendekatan ini, program tetap dapat melanjutkan eksekusi setelah exception ditangkap, seperti terlihat dari output “Out of the block”.

I.9 Latihan Defensive Programming (Multiple Catch Block)

I.9.A Langkah Kerja / Source Code

ExceptTest.java

```
import java.io.*;

public class ExceptTest {
```



```
public static void main(String args[]) {

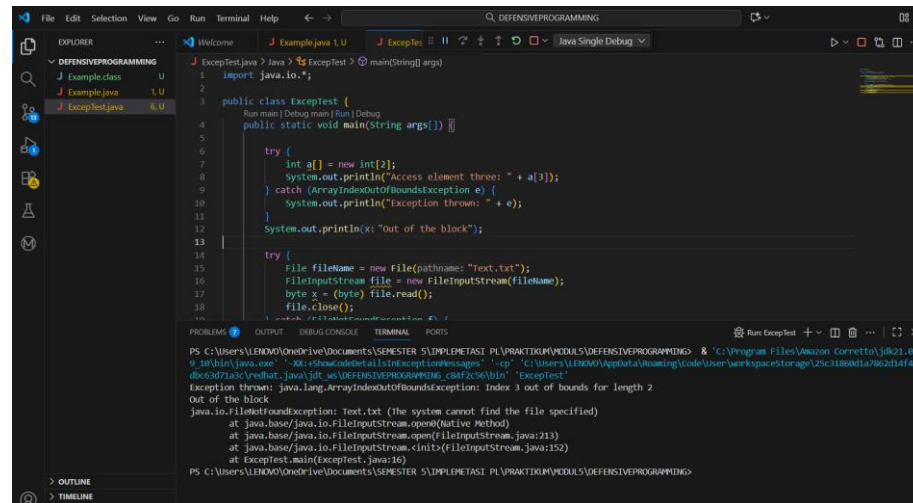
    try {
        int a[] = new int[2];
        System.out.println("Access element
three: " + a[3]);
    } catch (ArrayIndexOutOfBoundsException e)
    {
        System.out.println("Exception thrown:
" + e);
    }
    System.out.println("Out of the block");

    try {
        File fileName = new File("Text.txt");
        FileInputStream file = new
FileInputStream(fileName);
        byte x = (byte) file.read();
        file.close();
    } catch (FileNotFoundException f) {
        f.printStackTrace();
    } catch (IOException i) {
        i.printStackTrace();
    }

}

}
```

I.9.B Screenshoot



Gambar 1. 15 Multiple Catch Block

I.9.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menekankan penggunaan beberapa catch block untuk menangani berbagai jenis kesalahan secara spesifik. Pada contoh pertama, akses indeks array yang tidak valid ditangani oleh `ArrayIndexOutOfBoundsException`.

Pada contoh kedua, error terkait file seperti file tidak ditemukan atau IO error ditangani oleh catch block yang berbeda sesuai jenis kesalahannya. Pemecahan penanganan exception ke dalam beberapa blok membuat kode lebih aman dan mudah ditelusuri, karena setiap jenis error ditangani dengan cara yang tepat.

I.10 Latihan Defensive Programming (The Finally Block)

I.10.A Langkah Kerja / Source Code

ExcepTest2.java

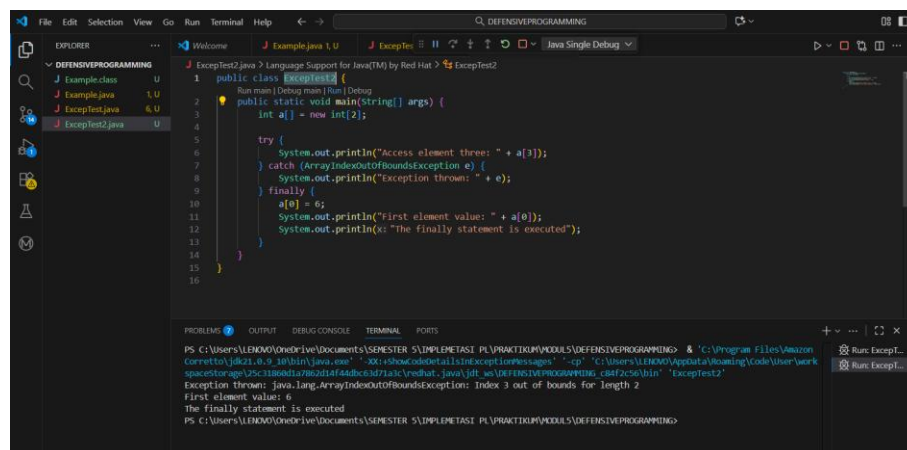
```
public class ExcepTest2 {  
    public static void main(String[] args) {  
        int a[] = new int[2];
```

```

        try {
            System.out.println("Access element
three: " + a[3]);
        } catch (ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Exception thrown:
" + e);
        } finally {
            a[0] = 6;
            System.out.println("First element
value: " + a[0]);
            System.out.println("The finally
statement is executed");
        }
    }
}

```

I.10.B Screenshoot



Gambar 1. 16 The Finally Block

I.10.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menunjukkan penggunaan blok finally untuk memastikan kode tertentu selalu dijalankan meskipun terjadi exception. Pada program, meskipun terjadi error saat mengakses indeks array, blok finally tetap dieksekusi untuk menetapkan nilai array dan menampilkan pesan tertentu.

Hal ini menunjukkan kekuatan finally dalam memastikan eksekusi perintah penting seperti cleanup, logging, atau pengembalian resource. Dengan memastikan bahwa operasi penting tetap dijalankan, blok finally menjadi elemen penting dalam defensive programming.

BAB II. TUGAS PRAKTIKUM

II.1 Tugas Clean Code (Naming)

II.1.A Langkah Kerja / Source Code

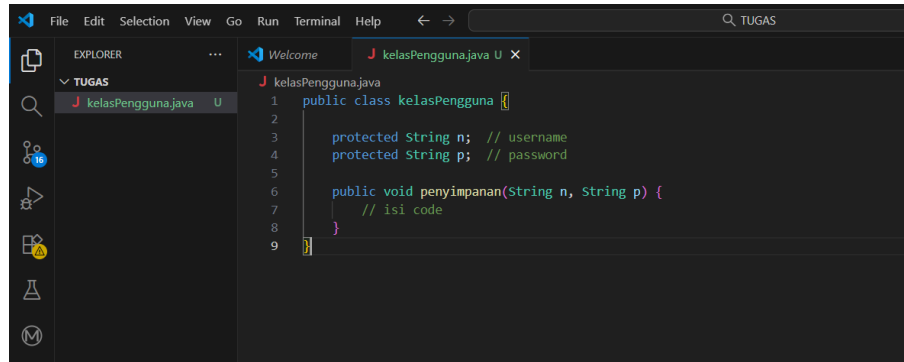
Sebelum

```
public class kelasPengguna {  
  
    protected String n; // username  
    protected String p; // password  
  
    public void penyimpanan(String n, String p) {  
        // isi code  
    }  
}
```

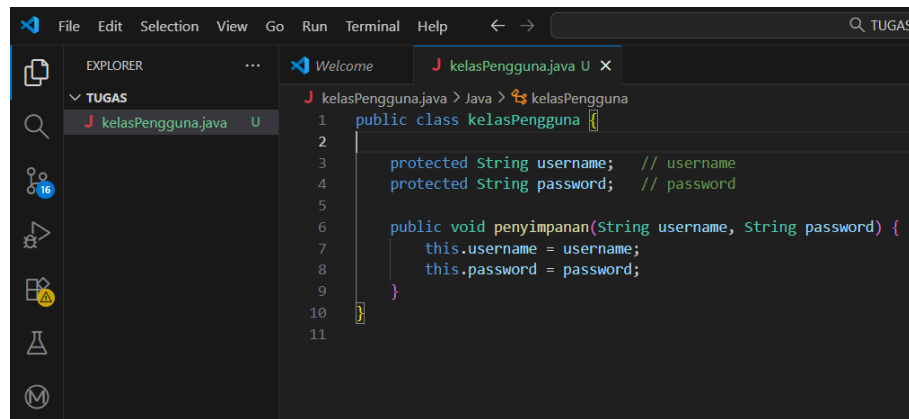
Sesudah

```
public class kelasPengguna {  
  
    protected String username; // username  
    protected String password; // password  
  
    public void penyimpanan(String username,  
String password) {  
        this.username = username;  
        this.password = password;  
    }  
}
```

II.1.B Screenshoot



Gambar 2. 1 Code Sebelum Diperbaiki



Gambar 2. 2 Code Sesudah Diperbaiki

II.1.C Analisis

Hasil analisis yang saya dapat pada Tugas ini yaitu mengubah penamaan variabel agar jelas dan bermakna untuk meningkatkan keterbacaan kode. Pada versi sebelum perbaikan, variabel bernama n dan p tidak memberikan konteks yang cukup mengenai data yang disimpan. Hal ini membuat kode sulit dipahami dan berpotensi membingungkan saat proses debugging maupun pengembangan lanjutan. Penggantian nama menjadi username dan password menjadikan maksud data jauh lebih eksplisit.

Selain itu, penggunaan parameter method yang selaras dengan penamaan atribut kelas mendukung prinsip clean code berbasis konsistensi. Penulisan

this.username = username memperjelas bahwa nilai yang diterima benar-benar disimpan sebagai state objek..

II.2 Tugas Clean Code (Formatting)

II.2.A Langkah Kerja / Source Code

Sebelum

```
public class Images
{fill(80,204,55);rect(100,0,200,400);
  var draw=function(){var numCircles = 6;for (var
i=numCircles; i>1; i--){var  radius=i*7;
  if (mouseX>100&&mouseX<300)
{fill(255,255,255);}else{fill(80,204,55);}ellipse(
mouse X,mouseY,radius,radius);}};
}
```

Sesudah

```
public class Images {

    fill(80, 204, 55);
    rect(100, 0, 200, 400);

    var draw = function() {
        var numCircles = 6;

        for (var i = numCircles; i > 1; i--) {
            var radius = i * 7;

            if (mouseX > 100 && mouseX < 300) {
```

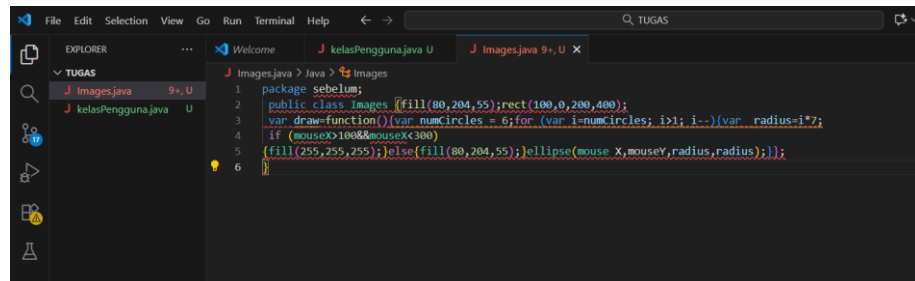
```

        fill(255, 255, 255);
    } else {
        fill(80, 204, 55);
    }

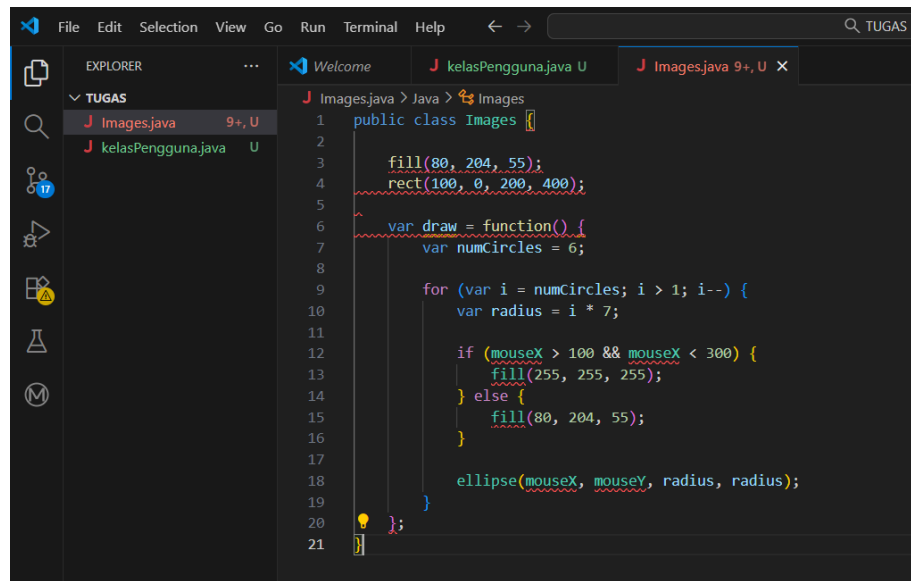
    ellipse(mouseX, mouseY, radius,
radius);
    }
};
}

```

II.2.B Screenshoot



Gambar 2. 3 Code Sebelum Diperbaiki



Gambar 2. 4 Code Sesudah Diperbaiki

II.2.C Analisis

Hasil analisis yang saya dapat pada Tugas ini yaitu memperbaiki formatting yang berperan penting dalam meningkatkan struktur dan keterbacaan kode. Pada versi sebelum perbaikan, kode terlihat padat, tidak memiliki indentasi yang konsisten, dan mencampurkan banyak instruksi dalam satu baris. Kondisi ini menyulitkan programmer untuk memahami alur logika terutama pada bagian perulangan dan kondisi. Perbaikan formatting membantu memisahkan logika sehingga kode menjadi lebih mudah dibaca.

Perubahan yang dilakukan memperlihatkan penggunaan indentasi, spasi, dan pemisahan blok kode secara tepat. Struktur yang jelas mempermudah identifikasi bagian tertentu seperti kondisi if, perhitungan radius, serta pemanggilan fungsi gambar..

II.3 Tugas Defensive Programming (nomor 3)

II.3.A Langkah Kerja / Source Code

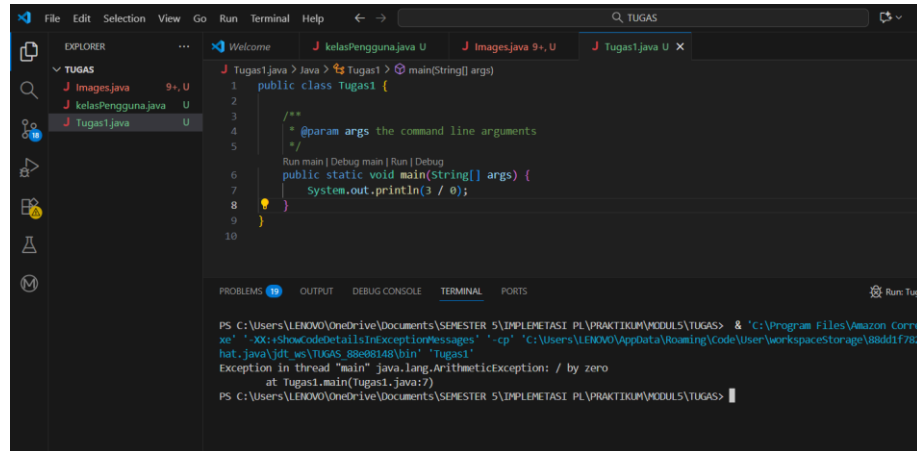
Sebelum

```
public class Tugas1 {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args) {  
        System.out.println(3 / 0);  
    }  
}
```

Sesudah

```
public class Tugas1 {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args) {  
        try {  
            System.out.println(3 / 0);  
        } catch (ArithmeticException e) {  
            System.out.println("Terjadi kesalahan:  
Tidak bisa membagi bilangan bulat dengan nol.");  
        }  
    }  
}
```

II.3.B Screenshoot



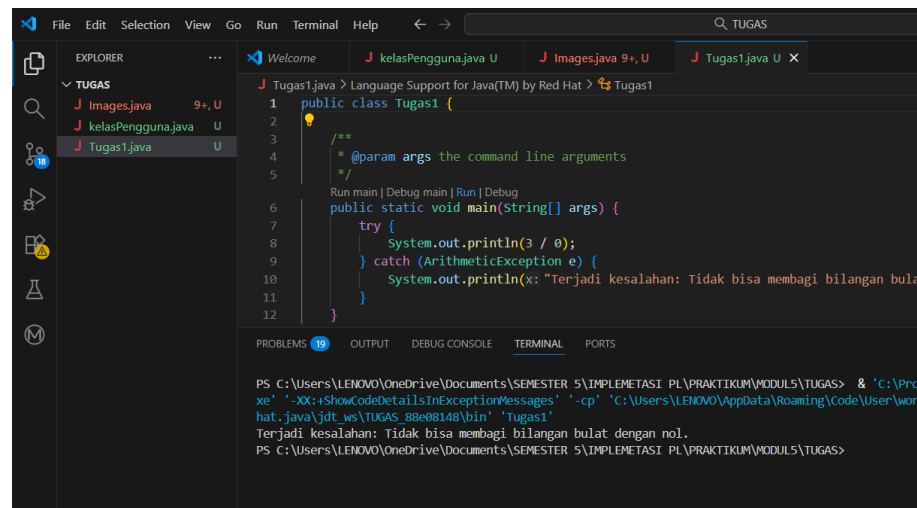
The screenshot shows an IDE with a Java file named `Tugas1.java`. The code is as follows:

```
1 public class Tugas1 {  
2  
3     /**  
4      * @param args the command line arguments  
5      */  
6     public static void main(String[] args) {  
7         System.out.println(3 / 0);  
8     }  
9  
10 }
```

The terminal output shows the command to run the program and the resulting exception:

```
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMENTASI PL\PRAKTIKUM\MODUL5\TUGAS> java -cp "C:\Program Files\Amazon Corretto\bin\java\jdt_ws\TUGAS_88e08148\bin" Tugas1  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at Tugas1.main(Tugas1.java:7)  
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMENTASI PL\PRAKTIKUM\MODUL5\TUGAS>
```

Gambar 2.5 Code Sebelum Diperbaiki



The screenshot shows the same IDE with the `Tugas1.java` file, but the code has been updated to include a try-catch block:

```
1 public class Tugas1 {  
2  
3     /**  
4      * @param args the command line arguments  
5      */  
6     public static void main(String[] args) {  
7         try {  
8             System.out.println(3 / 0);  
9         } catch (ArithmeticException e) {  
10             System.out.println(x: "Terjadi kesalahan: Tidak bisa membagi bilangan bulat dengan nol.");  
11         }  
12     }  
13 }
```

The terminal output now shows the program running successfully and printing the error message:

```
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMENTASI PL\PRAKTIKUM\MODUL5\TUGAS> java -cp "C:\Program Files\Amazon Corretto\bin\java\jdt_ws\TUGAS_88e08148\bin" Tugas1  
Terjadi kesalahan: Tidak bisa membagi bilangan bulat dengan nol.  
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMENTASI PL\PRAKTIKUM\MODUL5\TUGAS>
```

Gambar 2.6 Code Sesudah Diperbaiki

II.3.C Analisis

Hasil analisis yang saya dapat pada Tugas ini yaitu menambahkan blok try-catch untuk mencegah program mengalami crash ketika terjadi operasi tidak valid. Pada kode awal, pembagian dengan nol langsung menimbulkan exception dan menghentikan program tanpa memberikan informasi kepada pengguna. Ini merupakan contoh program yang tidak defensif terhadap

kemungkinan kesalahan logika atau input. Dengan menambahkan catch, program menjadi lebih terkendali.

Perubahan ini juga menunjukkan prinsip defensive programming yang memastikan bahwa error ditangani secara aman dan informatif. Semestinya menghentikan eksekusi, program memberikan pesan bahwa pembagian dengan nol tidak dapat dilakukan.

II.4 Tugas Defensive Programming (nomor 4)

II.4.A Langkah Kerja / Source Code

Sebelum

```
import java.io.*;
import java.util.Vector;

public class ListOfNumbers2 {

    private Vector victor;
    private static final int size = 10;
    public ListOfNumbers2() {
        victor = new Vector(size);
        for (int i = 0; i < size; i++)
            victor.addElement(new Integer(i));

        this.readList("infile.txt");
        this.writeList();
    }
    public void readList(String fileName) {
        String line = null;
        RandomAccessFile raf =
```

```

        RandomAccessFile(fileName, "r"); // ←
ini memang salah, tetap dibiarkan

        new
        while ((line = raf.readLine()) != null) {
            Integer i =
                new
Integer(Integer.parseInt(line));
            System.out.println(i);
            victor.addElement(i);
        }
        // tambahkan exception jika file tidak
ditemukan
        // tambahkan exception untuk output error
public void writeList() {
    PrintWriter out = null;
    out = new PrintWriter(
        new FileWriter("outfile.txt"));
    for (int i = 0; i < victor.size(); i++)
        out.println("Value at: " + i + " = "
            + victor.elementAt(i));
    // Tambahkan Exception jika array melebihi
batas
        // Tambahkan Exception untuk output error
        // Tambahkan Exception untuk exception
akhir:
        // menampilkan 'Closing Print Writer' jika
out tidak null
        // dan 'PrintWriter not open' jika out
null
    }

```

```
public static void main(String[] args) {  
    new ListOfNumbers2();  
}  
}
```

Sesudah

```
import java.io.*;  
import java.util.Vector;  
  
public class ListOfNumbers2 {  
  
    private Vector<Integer> victor;  
    private static final int size = 10;  
  
    public ListOfNumbers2() {  
        victor = new Vector<Integer>(size);  
        for (int i = 0; i < size; i++) {  
            victor.addElement(Integer.valueOf(i));  
        }  
  
        this.readList("infile.txt");  
        this.writeList();  
    }  
  
    public void readList(String fileName) {  
        String line = null;  
        RandomAccessFile raf = null;  
  
        try {
```

```

        raf = new RandomAccessFile(fileName,
        "r");

        while ((line = raf.readLine()) !=
        null) {

            Integer i =
            Integer.valueOf(Integer.parseInt(line));
            System.out.println(i);
            victor.addElement(i);
        }

    } catch (FileNotFoundException e) {
        System.err.println("Error: File " +
        fileName + " tidak ditemukan.");

    } catch (IOException e) {
        System.err.println("Error: Gagal
        membaca file. " + e.getMessage());

    } catch (NumberFormatException e) {
        System.err.println("Error: Format
        angka dalam file salah.");

    } finally {
        if (raf != null) {
            try {
                raf.close();
            } catch (IOException e) {
                System.err.println("Error saat
                menutup file input.");
            }
        }
    }
}

```

```

    }

    }

}

public void writeList() {
    PrintWriter out = null;

    try {
        out = new PrintWriter(new
FileWriter("outfile.txt"));

        for (int i = 0; i < victor.size();
i++) {
            try {
                out.println("Value at: " + i +
" = " + victor.elementAt(i));
            } catch
(ArrayIndexOutOfBoundsException e) {
                System.err.println(
                    "Error: Index array
melebihi batas pada index " + i
                );
            }
        }

    } catch (IOException e) {
        System.err.println("Error: Gagal
menulis ke outfile.txt");
    }
}

```



```

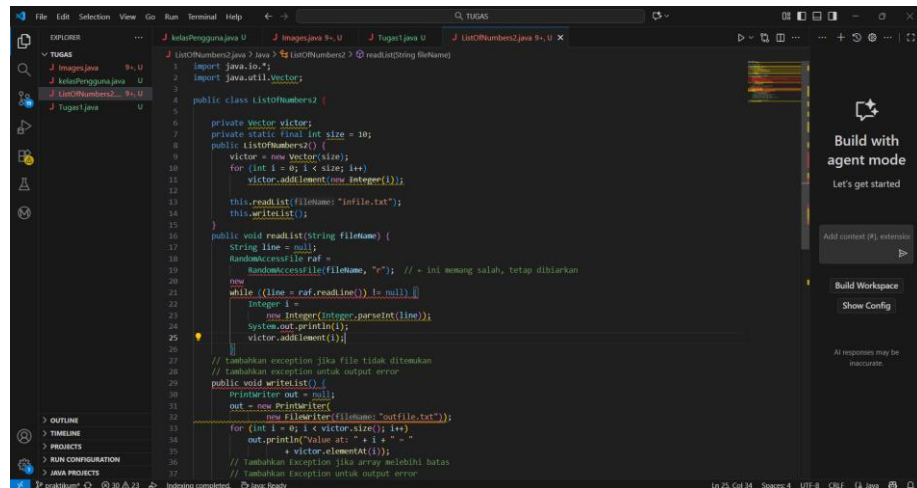
    } finally {
        if (out != null) {
            System.out.println("Closing
PrintWriter");

            out.close();
        } else {
            System.out.println("PrintWriter
not open");
        }
    }
}

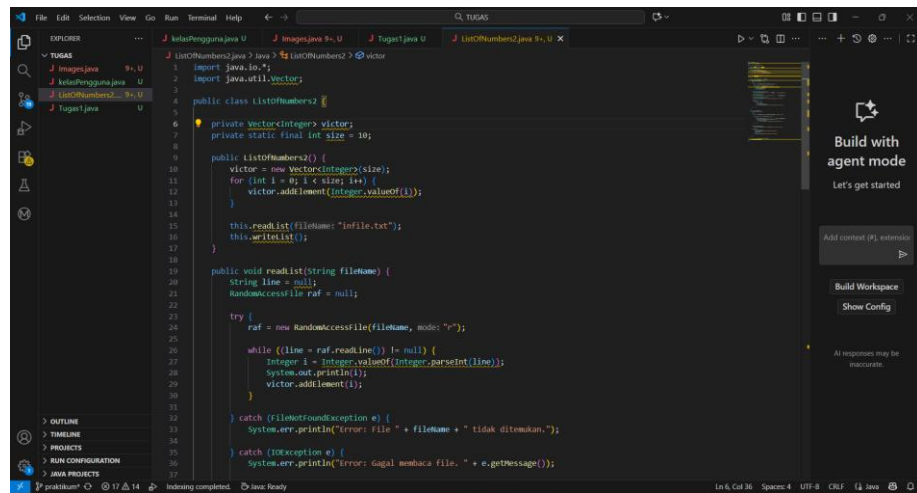
public static void main(String[] args) {
    new ListOfNumbers2();
}
}

```

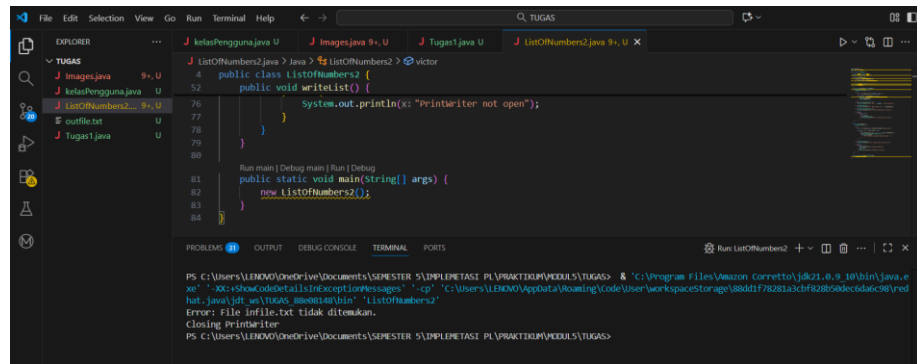
II.4.B Screenshoot



Gambar 2. 7 Code Sebelum Diperbaiki



Gambar 2. 8 Code Sesudah Diperbaiki



Gambar 2. 9 Ouput Code Setelah Diperbaiki

II.4.C Analisis

Hasil analisis yang saya dapat pada Tugas ini yaitu menerapkan defensive programming secara menyeluruh pada operasi file dan struktur data. Kode sebelum diperbaiki tidak memiliki penanganan kesalahan sama sekali, sehingga potensi error seperti file tidak ditemukan, format angka salah, atau error I/O dapat langsung menghentikan program. Ketidakhadiran mekanisme exception handling menjadikan program rapuh dan sulit diandalkan dalam situasi nyata. Dengan penanganan exception yang jelas, alur eksekusi menjadi lebih aman.

Setelah diperbaiki, program menambahkan berbagai jenis exception seperti FileNotFoundException, IOException, dan NumberFormatException untuk

menangani kemungkinan error dengan tepat. Selain itu, blok finally memastikan bahwa resource seperti file selalu ditutup meskipun terjadi kesalahan. Program juga menangani potensi `ArrayIndexOutOfBoundsException` saat membaca elemen vector.

BAB III. KESIMPULAN

Kesimpulan yang dapat saya ambil dalam pertemuan kelima pada Praktikum Implementasi Perangkat Lunak adalah dapat dipelajari prinsip-prinsip Clean Code yang berfokus pada peningkatan kualitas, keterbacaan, serta konsistensi penulisan program. Melalui latihan yang ada, dapat dipahami pentingnya penggunaan penamaan variabel yang jelas, struktur kode yang rapi, dan formatting yang teratur agar proses pengembangan, perbaikan, serta pemeliharaan kode menjadi lebih mudah. Penerapan Clean Code membantu menghasilkan program yang tidak hanya berjalan dengan benar, tetapi juga mudah dibaca dan berkelanjutan untuk pengembangan di masa depan.

Selain itu, konsep Defensive Programming juga dapat dipelajari pada pertemuan ini, yang menekankan bagaimana sebuah program harus mampu menghadapi berbagai kemungkinan error secara aman. Dengan menerapkan mekanisme seperti try-catch, validasi input, serta penggunaan blok finally, program dapat tetap berjalan stabil meskipun terjadi kondisi tak terduga seperti kesalahan perhitungan, format data yang salah, atau file yang tidak ditemukan.